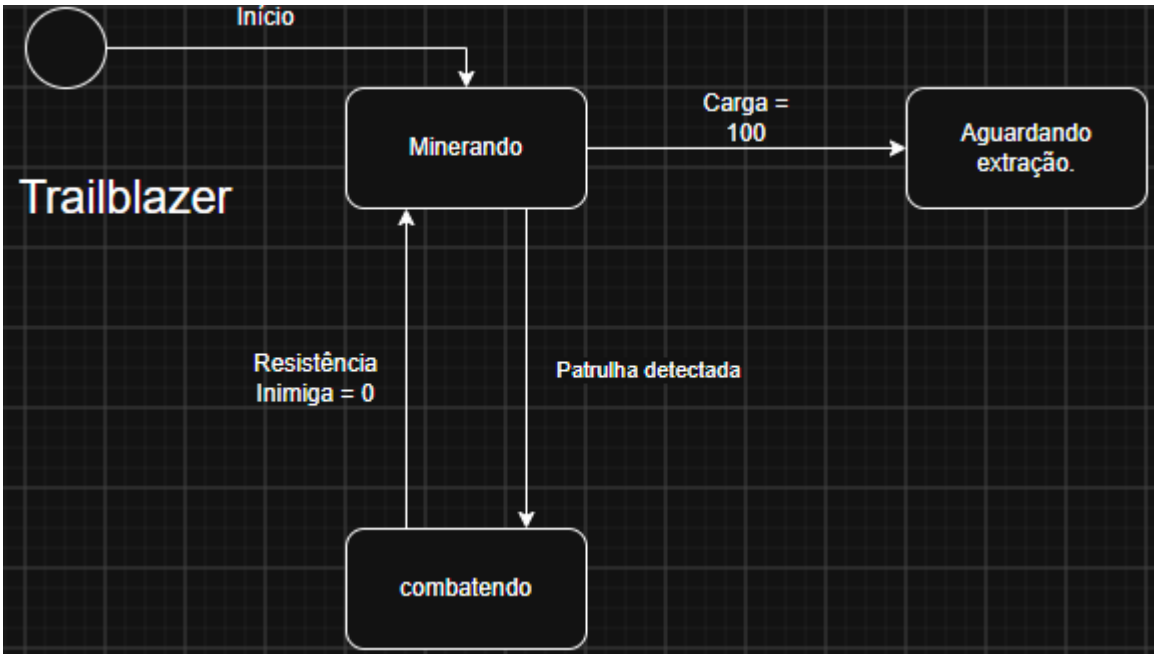
1. Visão geral do cenário: A empresa fictícia Raymond & Abramovich realiza uma operação de mineração em uma área sujeita à aproximação de patrulhas hostis. Um veículo terrestre coleta minério enquanto uma aeronave acompanha a segurança e organiza a retirada da carga. A simulação representa a missão por ciclos: o ambiente emite uma ameaça predeterminada, cada agente consulta as condições pertinentes e a conclusão ocorre após a extração. O objetivo técnico é demonstrar dois agentes com máquinas de estados independentes, cooperando por meio de regras definidas.
Dessa vez sem querer inventar moda e seguindo o exemplo.

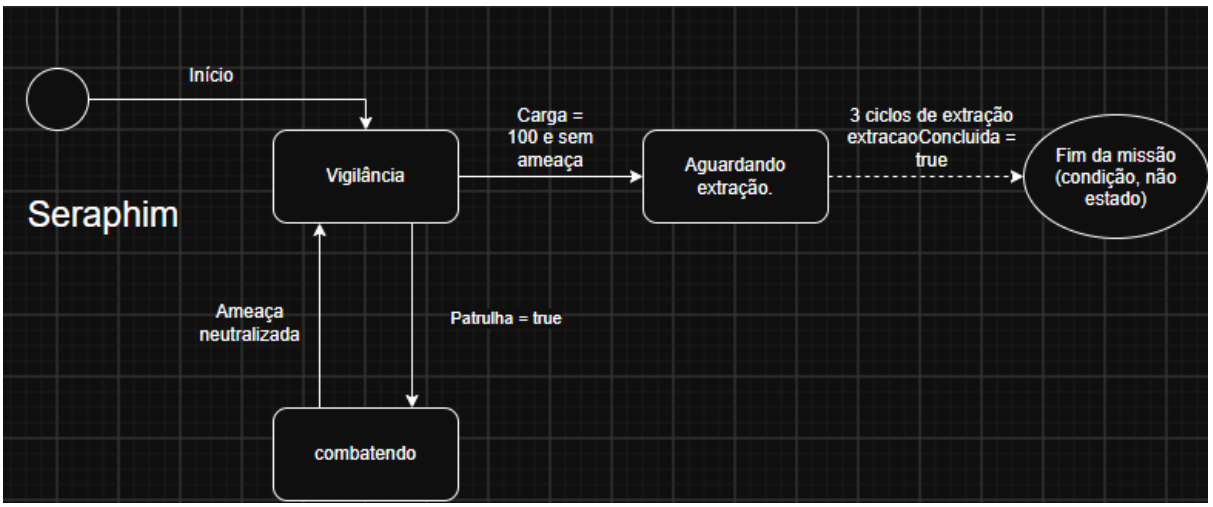
2. Visão geral dos agentes: Agente A — Trailblazer. Veículo terrestre de mineração com integridade inicial de 100 pontos, carga inicial zero e capacidade de 100 unidades. Seus três estados são Minerando (coleta 20 unidades por ciclo), Combatendo (enfrenta uma patrulha com resistência inicial de 3) e AguardandoExtração (aguarda a aeronave com a carga completa). Ao entrar em combate, preserva a carga já obtida.

Agente B — Seraphim. Aeronave de vigilância e retirada que mantém uma referência ao Trailblazer. Seus três estados são Vigilância (monitora ameaça e carga), Suporte (acompanha o confronto) e Extração (executa procedimento de três ciclos). Duas flags booleanas representam a detecção de patrulha e a conclusão da extração.

3. Implementação: Foi aplicado o padrão State com a interface genérica State e a classe abstrata AbstractState. Cada estado concreto implementa execute() e define, quando necessário, ações de enter() e leave(). A interface Agente padroniza update(), setState() e printStats(); o gerenciador StateMachine mantém os agentes e conduz o loop. São utilizadas somente classes do Java padrão, sem aleatoriedade e sem bibliotecas externas.

4. Diagramas de estados:





5. Tabelas de regras por agente:

Trailblazer:

Estado	Entrada: enter()	Execução: execute()	Saída: leave()	Regra de transição
Minerando	Liga perfuratrizes; anuncia início.	Se carga < 100, adiciona 20 unidades e imprime o total.	Desliga perfuratrizes.	Carga ≥ 100 → AguardandoExtração. Se a Seraphim detectar patrulha, solicita Combatendo.
Combatendo	Define resistência da patrulha = 3; ativa defesa.	Se operacional: recebe 5 de dano, então reduz a resistência inimiga em 1.	Desativa sistemas de defesa.	Resistência inimiga ≤ 0 → Minerando; integridade = 0 → sem novo estado (falha do loop).
AguardandoExtração	Zera ciclos de espera e anuncia retirada.	Incrementa contador e registra espera.	Registra saída, caso haja.	Não possui transição normal; a Seraphim conclui a missão.

Seraphim:

Estado	Entrada: enter()	Execução: execute()	Saída: leave()	Regra de transição
Vigilância	Ativa sensores.	Testa se Trailblazer está operacional; depois a flag de patrulha; depois a carga.	Encerra vigilância e ajusta sensores.	Patrulha = true → Suporte e aciona combate no Trailblazer. Senão, carga ≥ 100 → Extração.
Suporte	Ativa protocolo de cobertura.	Enquanto o Trailblazer estiver em combate, informa acompanhamento.	Desativa protocolo de cobertura.	Trailblazer operacional e fora de combate → limpa flag e retorna a Vigilância.
Extração	Zera contador; inicia deslocamento.	1º ciclo: aproximação; 2º: acoplamento/transferência; 3º: conclui extração.	Registra saída, caso haja.	Após 3 ciclos, extracaoConcluida = true e o loop é encerrado.

6. Variáveis, intervalos e limiares:

Trailblazer e seus estados:

Variável (tipo)	Valor inicial / intervalo	Uso / limiar
integridade (int)	100; intervalo 0–100	-5 por ciclo de combate; 0 = veículo não operacional.
carga (int)	0; intervalo 0–100	+20 por ciclo de mineração; ≥ 100 = aguardar extração.
capacidadeCarga (int)	100; constante	Limite superior de carga.
resistenciaPatrulha (int)	3 ao entrar; 0–3	-1 por ciclo de combate; ≤ 0 = vitória.
ciclosDeEspera (int)	0 ao entrar; ≥ 0	Conta atualizações em AguardandoExtração.
state (State<Trailblazer>)	Minerando; referência	Estado atual, alterado por setState().

Seraphim e seus estados:

Variável (tipo)	Valor inicial / intervalo	Uso / limiar
patrulhaDetectada (boolean)	false; true/false	true no ciclo 3 → Suporte; false após a vitória.
extracaoConcluida (boolean)	false; true/false	true no 3º ciclo de Extração → fim do loop.
ciclosExtracao (int)	0 ao entrar; 0–3	1/2/3 representam os passos da extração.
trailblazer (Trailblazer)	Referência ao agente A	Consulta integridade, carga e estado de combate.
state (State<Seraphim>)	Vigilância; referência	Estado atual da aeronave.
ciclo (int)	0; 0–20	Contador global. Patrulha no ciclo 3; teto 20.

7. Estrutura de código e responsabilidades:

Arquivos	Responsabilidade
State.java; AbstractState.java	Contrato genérico com enter/execute/leave e base que guarda o agente.
Agente.java	Contrato compartilhado: update, setState, printStats.
Trailblazer.java; Seraphim.java	Dados e troca de estados de cada agente; Seraphim referencia Trailblazer.
EstadoMinerando.java; EstadoCombatendo.java; EstadoAguardandoExtracao.java	Três comportamentos específicos do Trailblazer.
EstadoVigilancia.java; EstadoSuporte.java; EstadoExtracao.java	Três comportamentos específicos da Seraphim.
StateMachine.java; Main.java	Loop/eventos determinísticos e ponto de entrada da aplicação.

8. Resumo:

Foram implementados dois agentes, cada um com três estados próprios (superando os mínimos de 3 e 2). Os estados oferecem os três métodos do padrão State, a StateMachine gerencia o loop, os gatilhos são contadores/flags/limiares, os logs tornam as transições observáveis e o projeto utiliza apenas Java padrão.